Trafasa Technology – Round 1 Assessment (Take-Home)

Name: **Rajarsi Roy Chowdhury**

College: **Narula Institute of Technology**

Department: **Computer Science Engineering (AI & ML)**

Email: rcrajarsi22@gmail.com

Contact No.: **(+91) 6290350360**

Git Assignment Repository Link: GitHub Repo Link

Section A – Core Skills

Question 1 – Python Fundamentals & Logic

Task:

Write a Python function named fetch_vendor_data(vendor_ids) that:

1. Accepts a list of vendor IDs.
2. Simulates a slow API call by sleeping 0.5 seconds for each vendor ID.
3. Fetches data from a mock database (provided as a Python dictionary):

```
mock_db = {
    "V001": {"name": "Alpha Corp", "rating": 4.5},
    "V002": {"name": "Beta Ltd", "rating": 3.8},
    "V004": {"name": "Delta Inc", "rating": 4.9},
}
```

4. Gracefully handles missing vendor IDs (logs them, but does not crash).
5. Returns a dictionary of successfully retrieved vendors

Solution

Code:

```
import time
import logging

mock_db = {
    "V001": {"name": "Alpha Corp", "rating": 4.5},
    "V002": {"name": "Beta Ltd", "rating": 3.8},
    "V004": {"name": "Delta Inc", "rating": 4.9},
}

def fetch_vendor_data(vendor_ids):

    fetched_vendors = {}
```

```
    for vendor_id in vendor_ids:
        time.sleep(0.5)

        if vendor_id in mock_db:
            fetched_vendors[vendor_id] = mock_db[vendor_id]
        else:
            logging.warning(f"Vendor ID ‘{vendor_id}’ Not Found!")

    return fetched_vendors


if __name__ == '__main__':
    vendor_ids = ["V001", "V002", "V003", "V004", "V005"]
    result = fetch_vendor_data(vendor_ids)
    print("Fetched Vendor Data: ")
    for vendor_id, details in result.items():
        print(f"{vendor_id} : {details}")
```

Output:

```
WARNING:root:Vendor ID 'V003' Not Found!
WARNING:root:Vendor ID 'V005' Not Found!
Fetched Vendor Data:
V001 : {'name': 'Alpha Corp', 'rating': 4.5}
V002 : {'name': 'Beta Ltd', 'rating': 3.8}
V004 : {'name': 'Delta Inc', 'rating': 4.9}
```

Bonus Explanation

The Process of fetching the vendor data is an I/O-Bound Operation, i.e., simulated API calls, so the requests can be executed concurrently instead of sequentially. Python provides a way to achieve concurrent executions using the `concurrent.futures` module with `ThreadPoolExecutor`. `ThreadPoolExecutor` creates a pool of worker threads, where each thread can fetch data for a different vendor simultaneously. Running multiple fetches concurrently significantly reduces the overall execution time.

Question 2 – SQL & Data Modeling

Task:

You are designing a procurement database. One workflow involves Purchase Orders (POs) and Goods Received Notes (GRNs).

1. **Design:** Draw or describe an Entity-Relationship (ER) diagram for PO and GRN. Clarify their relationship (e.g., one-to-many: a PO can have multiple GRNs, but each GRN belongs to exactly one PO). Include key attributes (like po_id, item_code, quantity_ordered, etc.) that you consider important.
2. **SQL Query:** Write a SQL query to compute the total quantity received for a given PO_ID = 'PO-2024-001'. Assume the following simplified tables:
    - po_items (po_id, item_code, quantity_ordered)
    - grn_items (grn_id, po_id, item_code, quantity_received)

Solution

1. Entity Relationship (ER) Diagram

Entities:

(i) ***Purchase_Order*** *(po_id, supplier_name, order_date, status)*
(ii) ***PO_Items*** *(po_id, item_code, quantity_ordered)*
(iii) ***Goods_Received_Note*** *(grn_id, po_id, received_date)*
(iv) ***GRN_Items*** *(grn_id, po_id, item_code, quantity_received)*

Relationships:

(i) ***Purchase_Order → Goods_Received_Note (1 to Many)***

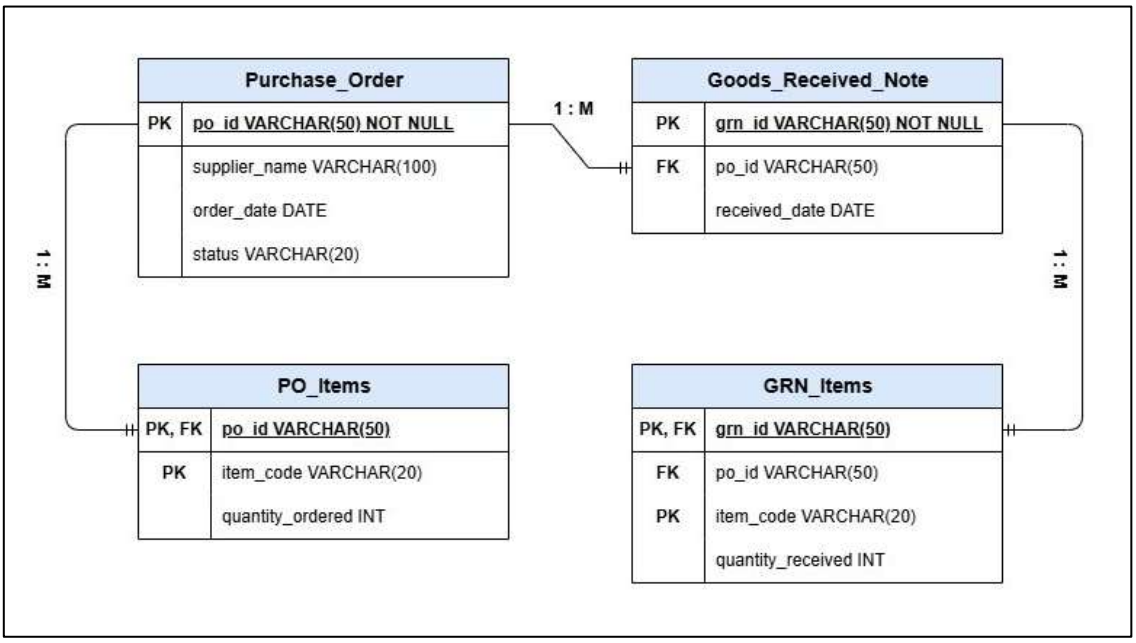
One Purchase Order can have many GRNs. One GRN belongs to exactly one Purchase Order.

(ii) ***Purchase_Order → PO_Items (1 to Many)***

One Purchase Order can contain multiple items. Each PO item belongs to exactly one Purchase Order.

(iii) ***Goods_Received_Note → GRN_Items (1 to Many)***

One Goods Received Note (GRN) can contain multiple received items. Each GRN Item belongs to exactly one Goods Received Note.



2. SQL Query

```
SELECT SUM(quantity_received) AS Total_Quantity_Received
FROM grn_items
WHERE po_id = 'PO-2024-001';
```

Question 3 – REST APIs & Git

Task:

1. **API Design:** Design a RESTful endpoint /api/payments/initiate that the frontend calls to start a payment. Provide:
    - The expected JSON request body.
    - A sample JSON success response.
2. **Git:** You are on a feature branch feature/add-whatsapp-integration. A colleague has just merged a critical bug fix into develop. Write the Git commands you would use to bring those changes into your branch without performing a merge (i.e., rebasing or cherry-picking). Explain your choice.

Solution

1. API Design

Endpoint:

```
POST /api/payments/initiate
```

Request Body (JSON):

```
{
    “order_id”: “ORD-2026-001”,
    “amount”: 2500.00,
    “currency”: “INR”,
    “payment_method”: “UPI”,
    “customer_id”: “CUST-101”
}
```

Success Response (Code: 201 Created)

```
{
    “payment_id”: “PAY-2026-001”,
    “status”: “Pending”,
    “message”: “Payment Initiated Successfully!”
}
```

2. Git

Commands:

```
git checkout develop
git pull origin develop
git checkout feature/add-whatsapp-integration
git rebase develop
```

Explanation:

I would use git rebase because, the required bug fix has already been merged into the develop branch. Rebasing my feature branch onto the updated develop branch would incorporate the latest changes without having to create a merge commit.

While git cherry-pick is more suitable for when only a specific commit is needed to be copied from another branch.

In this scenario, since the latest develop branch needs to be incorporated, git rebase would be the best choice.

Section B – Specialized Tracks

Track: AI/Data Engineering

Question 4 – LLMs, RAG & Agents

Task:

The JD describes an AI agent that parses an RFQ, pulls vendor context from a vector store, and scores quotes.

1. **RAG Pipeline:** Draw or describe the high-level steps of a Retrieval-Augmented Generation (RAG) pipeline that would supply the vendor context for this agent.
2. **Agentic AI:** What is the difference between simply calling an LLM (e.g., gpt-4) and building an "Agentic AI" as described in the JD? Explain the concepts of tool use, memory, and human-in-the-loop.
3. **Prompt Engineering:** Write a system prompt for the LLM that scores a vendor quote based on price, delivery_time, and compliance. The prompt must instruct the LLM to output a JSON object containing the score and a brief justification.

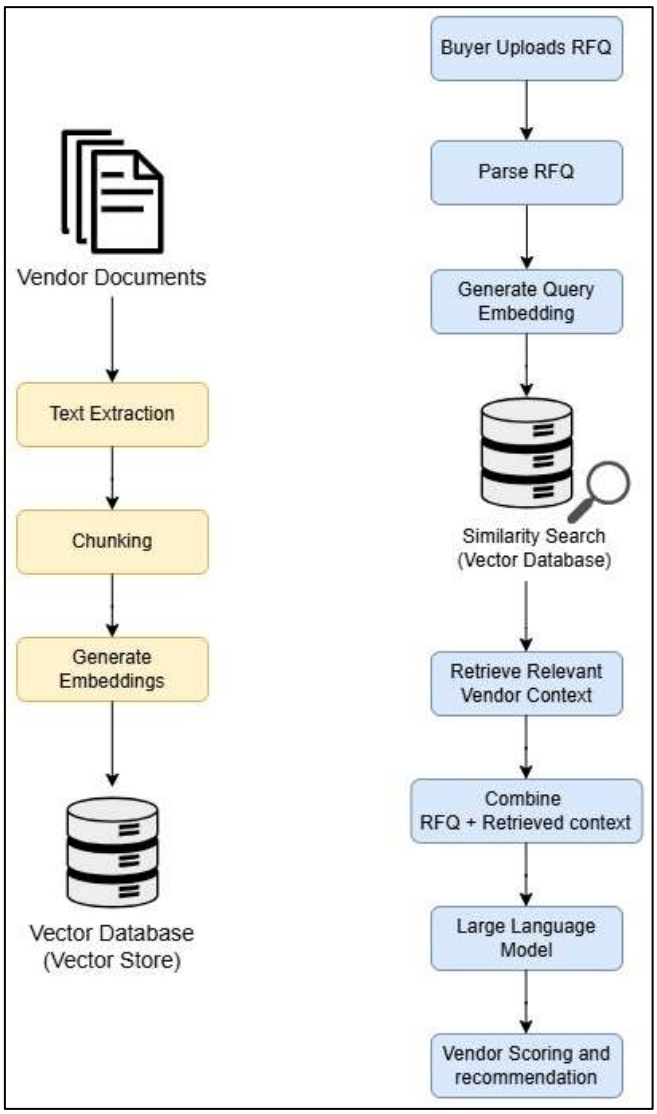
Solution

1. RAG Pipeline

Explanation:

The RAG Pipeline begins by collecting vendor-related documents like vendor profiles, historical quotations, delivery records and performance ratings. These documents are parsed and text is extracted from them, divided into smaller chunks (Chunking), and transformed into vector embeddings using an embedding module. The embeddings are finally stored in a vector database.

When a buyer uploads an RFQ, the system converts the RFQ into an embedding and performs a similarity search on the vector database to fetch the most relevant vendor context. The retrieved context combined with the RFQ is passed to the LLM, which generates an informed response and by scoring vendors and recommending suitable suppliers.



2. Agentic AI

A Large Language Model (LLM) generates responses based on a single prompt and does not independently plan tasks or interact with external systems. In contrast, an Agentic AI plans multiple steps, uses external tools and APIs, maintains memory across interactions and decides which action to perform, to accomplish a goal.

So, calling an LLM would provide us with a simple and generic response, while an Agentic AI has the ability to act, not just respond.

Tool Use:

Tool Use enables Agentic AI systems to interact with external resources such as databases, APIs, vector stores, document parsers, etc., to retrieve real-time information.

Memory:

Memory enables Agentic AI systems to retain relevant information from previous interactions or workflow steps, which helps the AI Agent to maintain context and make more informed decisions.

Human-in-the-Loop:

Human-in-the-Loop is a feature that allows human intervention during critical or low confidence responses by the AI Agent. This reduces errors impacting business decisions.

3. Prompt Engineering

System Prompt:

```
You are an AI Procurement Assistant responsible for evaluating vendor quotations.

Your task is to score each vendor quote based only on the following criteria:

1. Price – Lower prices should receive higher scores.
2. Delivery Time – Shorter delivery time should receive higher scores.
3. Compliance – Vendors that fully meet all specified requirements should receive higher
   scores.

Instructions:

-  Evaluate the vendor objectively using only the provided information.
-  Do not make assumptions or invent missing data.
-  If any required information is missing, mention it in the justification.
-  The final score must be between 0 and 100.
-  Return only a valid JSON Object, without any additional text or explanation.

Output Format:

{
    “score”: <number between 0 and 100>,
    “justification”: “<brief explanation for the assigned score>”
}
```

Question 5 – Data Pipelines & Evaluation

Task:

You are building an IDP (Intelligent Document Processing) pipeline to extract data from invoices.

1. **Data Extraction:** Describe your approach to extracting structured fields (e.g., invoice_number, invoice_date, total_amount, vendor_name) from PDF invoices that have varying layouts. What are the main challenges and how would you address them?

2. **Model Quality:** Once deployed, how would you measure the accuracy of your extraction over time? The JD mentions "tracking prediction drift" – what metrics and monitoring strategies would you implement?

3. **Fallback Logic:** The JD explicitly states: "so a model failure doesn't break a live procurement process." Design a robust fallback strategy for your extraction pipeline. What should happen if the LLM fails to parse the document or returns unreliable output?

Solution

1. Data Extraction

Code:

```
import re
import json

def extract_text():
    """
    Simulating text extraction from a PDF invoice.
    """
    return """
    Invoice Number: INV-2026-001
    Invoice Date: 08-07-2026
    Vendor: ABC Steel Pvt Ltd
    Total Amount: 48490
    """

def llm_extract_fields(text):
    """
    Simulating an LLM extracting structured fields.
    """

    data = {
        "invoice_number": re.search(r"Invoice Number:\s*(.+)", text).group(1),
        "invoice_date": re.search(r"Invoice Date:\s*(.+)", text).group(1),
        "vendor_name": re.search(r"Vendor:\s*(.+)", text).group(1),
        "total_amount": re.search(r"Total Amount:\s*(.+)", text).group(1)
    }

    return data

def main():
    invoice_text = extract_text()
    extracted_data = llm_extract_fields(invoice_text)

    print(json.dumps(extracted_data, indent=4))

if __name__ == "__main__":
    main()
```

Output:

```
{
    "invoice_number": "INV-2026-001",
    "invoice_date": "08-07-2026",
    "vendor_name": "ABC Steel Pvt Ltd",
    "total_amount": "48490"
}
```

2. Model Quality

After deployment, I would continuously monitor the extraction model to ensure it maintains high accuracy. The extracted invoice would be periodically compared against manually verified data to evaluate performance.

Metrics Used:

- Field Accuracy: Percentage of correctly extracted fields like invoice number, invoice date, etc.
- Confidence Score: Average confidence of the extracted values. Predictions with low confidence can be flagged for manual review.
- Prediction Drift: Monitor changes in the model's predictions over time.
- Error Rate: Track the percentage of invoices that failed extraction.

Monitoring Strategies:

- Maintaining logs of all extraction results and confidence scores.
- Periodic comparison of predictions with verified invoice data.
- Setting alerts when accuracy drops below a specified threshold.
- Retraining the model using newly collected samples during significant drift in prediction.

3. Fallback Logic

Ensuring that the procurement process continues even if the AI model fails would require a robust fallback mechanism.

Fallback Strategy:

- Firstly, the LLM attempts to extract the required invoice fields from the document.
- The extracted data is then validated for completeness and confidence.
- After successful extraction, the data proceeds to the procurement workflow.
- If the LLM fails, or produces low-confidence results, the invoice is flagged for manual review automatically.
- The original invoice and extracted data are retained, and the corrected data is used to improve or retrain the model.

Section C – Why Trafasa?

1. The JD states that the platform serves "the ~90% who still manage purchasing over WhatsApp, spreadsheets, or nothing at all." Why do you believe this is a meaningful problem, and what excites you about building a solution for it?

2. The team is small and interns "work on the actual product." Describe a time when you took ownership of a project from start to finish or went beyond your assigned tasks to make something work. What was the outcome?

Solution

1. I believe this is a meaningful problem as many small and medium-sized businesses still rely on manual processes like WhatsApp messages and spreadsheets to manage and administrate procurement. These methods are time consuming, error-prone and make it difficult to track purchase orders, vendor information, and invoices efficiently. Building a platform that digitizes and automates these workflows can improve productivity, reduce errors, and help businesses make faster and more informed decisions.

What excites me about Trafasa is that it is not just building another procurement tool – it is solving a real-world problem that many businesses face every day. I like the idea of working on a product where the technology directly improves how people work and creates measurable value for its users. As a fresher, I would love the opportunity to apply my technical knowledge, learn from an experienced team, and contribute to building a product that makes a real impact.

2. During my ELT project, I took ownership of building an end-to-end ELT pipeline using Python, SQL Server, and Microsoft Power BI. I collected the dataset, performed data extraction using Python and loaded the data into SQL Server for transformation, and wrote SQL Queries for analysis. Finally, I developed interactive Power BI dashboards to visualize key business insights.

   Throughout the project, I was responsible for every stage, from designing the data pipeline to validating the final dashboard. This experience improved my problem-solving skills and taught me how different components work together to deliver a complete solution. It also strengthened my confidence in taking responsibility for a project from start to finish.

Submission Checklist

☑ All answers are typed clearly in a single PDF.

☑ Code for Q1 and Q4 (Platform) / Q5 (AI) is included and linked to a repository/gist.

☑ You have chosen one track and answered that section only